

Drift-Sense

AI-Powered Navigation-Error Recovery for Wafer Inspection Tools

A Siamese dense-correlation deep-learning localizer with a global-context head, trained on a physically-grounded synthetic DRAM-SEM generator, that locates a 100x reference patch inside a 10x search image to sub-pixel accuracy.

Team: KryonixNova

Members: [MEMBER NAME - ROLE] | [MEMBER NAME - ROLE] | [MEMBER NAME - ROLE]

College: [COLLEGE NAME]

Contact: [CONTACT EMAIL / PHONE]

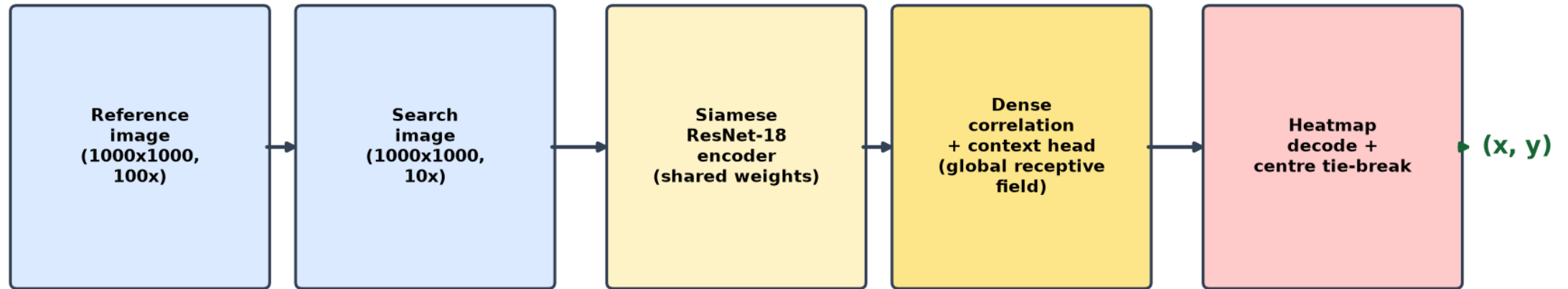
SEMI x IESA Hackathon 2026 -- Applied Materials Problem Statement

Problem Understanding

Why navigation-error recovery matters in wafer inspection

- Semiconductor wafers contain highly repetitive, periodic microscopic structures (DRAM cell arrays: word lines, bit lines, storage-node contacts).
- Inspection tools must repeatedly revisit the same relative location across dies -- stage drift, vibration, and thermal effects cause the tool to land away from the intended site.
- Recovery is visual: given a previously-captured 100x close-up reference image, find the same structure inside a wider, lower-resolution 10x search image and return its centre coordinates.
- The core difficulty is not just matching -- it's disambiguation: a small reference patch can look near-identical to dozens of other lattice repeats. Simple template matching (e.g. classical ZNCC) frequently locks onto the wrong repeat with high confidence.
- Getting this wrong means the tool inspects, measures, or processes the wrong die location -- a real cost in a production inspection pipeline.

Proposed End-to-End Workflow



- `generate_dataset.py` builds physically-grounded synthetic reference/search pairs with recorded ground truth (see slide 4).
- `localize.py` runs the trained model on a pair or an evaluator-supplied batch, no source-code edits between modes.
- `scripts/validation_report.py` scores the pipeline across noise x geometry conditions and produces the required accuracy/runtime/failure-case report.

Synthetic-Data Design

DRAM-style structures, chosen and validated end-to-end

- Architecture: DRAM-style (folded-bitline 1T-1C array) -- word lines, bit lines, and a checkerboard of storage-node contacts (one per 2 cells, matching real folded-bitline layouts, not a naive full grid).
 - Non-proprietary by construction: cell_pitch_nm, word/bit-line width, and contact size are free parameters. No specific process node is implied -- backed by public 1T-1C DRAM literature (see slide 5's citations).
- 6 distinct DRAM presets (dram_1x/dense/loose/wide/compact/legacy) drawn per mat, tiled in independent m x n grids separated by routing-strip material for realistic die-layout diversity.
- Every pair records: random seed, architecture, transformations (scale/rotation), noise settings, and true target-centre coordinates -- full per-pair metadata for reproducibility.
 - FinFET pattern code (src/patterns/finfet.py) exists in the repository but was not wired into training for this submission -- DRAM was the chosen, validated architecture (see Limitations, slide 11).

Noise, Blur, Scale & Rotation Modelling

Every augmentation choice backed by a public source

- Imaging degradation (two severity profiles, normal/harsh): Poisson shot noise (dose-driven electron-count proxy), detector noise, astigmatism, vignetting, barrel distortion, charging streaks, beam-drift jitter, speckle, salt-and-pepper.
- Scale robustness: reference crops jittered 0.9x-1.1x (an effective 9:1-11:1 relationship against the nominal 10:1), matching the spec's stated test range.
- Rotation robustness: +/-1-2 degree rotation applied to reference crops post-imaging, simulating real capture calibration/stage drift -- ground truth is unaffected, only the reference's content is perturbed.
- Citations:
 - imec, "DRAM peripheral transistors technology platform" -- 1T-1C cell structure.
 - SemiAnalysis, "The Memory Wall: Past, Present, and Future of DRAM" -- word/bit-line array structure.
 - "Correction of SEM Imaging Artifacts..." (Experimental Mechanics, PMC) -- drift/charging-driven image distortion.
 - "SEM Image SNR Monitoring for Micro-Nanomanipulation" (HAL) -- Poisson shot-noise physical basis.
 - Two arXiv papers on training-time rotation/scale augmentation for deep template matching and orientation invariance in feature matchers.

Localization Method & Decision Rule

Deep learning: dense correlation + global context

- Siamese ResNet-18 encoder (destrided, no pretrained weights) -- shared weights encode both the reference and search images into feature maps.
- Dense depthwise cross-correlation slides the reference feature template across the search feature map, producing a correlation heatmap over candidate locations.
- 9-layer dilated convolution context head with a global receptive field -- this is what a purely local encoder/correlation step cannot do: resolve which of several identical-looking lattice repeats is the true match, using mat/die boundaries and aperiodic imaging noise as disambiguating context.
- Training targets: Gaussian heatmap peak + sub-cell offset regression. Loss: focal heatmap loss + offset L1 + a hard-negative margin term.
- Decode: NMS peak extraction, with an explicit centre-tiebreak rule -- if multiple valid matches exist, the candidate closest to the search image's centre is chosen, exactly matching the spec's tie-break requirement.
- Confidence is a peak margin, not a raw score -- the classical ZNCC baseline scored misleadingly high (0.9+) even when wrong ~35% of the time, so absolute score alone doesn't separate right from wrong for this task.

Implementation & Execution

Runs without manual edits, on a fresh machine

Setup:

```
pip install -r requirements.txt
```

Generate a dataset (architecture, sample count, output dir all parameterized):

```
python generate_dataset.py --split test --num-samples 30 --output-dir ./output
```

Run inference on a single pair (the script Applied Materials will run):

```
python localize.py --reference ref.png --search search.png --xy-only
```

Run inference on an evaluator-supplied batch manifest:

```
python localize.py --manifest manifest.csv --output predictions.csv
```

Reproduce the full validation report (accuracy, runtime, failure case):

```
python scripts/validation_report.py --checkpoint checkpoints/production_v3/best.pt --n-per-condition 50 --out-dir results
```

Generator (generate_dataset.py) and inference (localize.py) are separate, independently-runnable Python scripts -- not a single notebook.

Experiment Setup & Test Diversity

- scripts/validation_report.py: 200 independently-generated pairs (50 per condition) across all 4 imaging-noise x geometric-profile combinations (normal/normal, harsh/normal, normal/drift, harsh/drift).
- results/sample_dataset/: a committed, concrete 32-pair example (8 per condition) run end-to-end through generate_dataset.py -> localize.py, combined ground-truth + prediction CSV.
- Diversity covered per the spec's validation requirements:
 - Noise levels: normal and harsh imaging-noise profiles.
 - Target positions: random reference-crop placement per canvas, every sample.
 - Scale: 0.9x-1.1x reference jitter (9:1-11:1 effective ratio).
 - Rotation: +/-1-2 degree reference rotation.
 - 6 DRAM presets, multiple m x n mat-tiling configurations.
- All test samples drawn from a canvas-disjoint seed range, never overlapping training/validation seeds.

Results: Accuracy, Error Statistics & Runtime

Condition	n	Mean err (px)	Median err (px)	pass@5px	pass@4px	pass@2px	pass@1px
normal / normal	50	0.64	0.57	1.000	1.000	1.000	0.860
harsh / normal	50	4.32	3.59	0.640	0.540	0.340	0.220
normal / drift	50	0.67	0.59	1.000	1.000	1.000	0.860
harsh / drift	50	4.28	3.44	0.640	0.540	0.340	0.220

- Pooled: n=200, mean error = 2.47px.
- Runtime: median 22.0ms per pair (model.predict() only, wall-clock via time.perf_counter()) on an NVIDIA GeForce RTX 5060 Ti, Python 3.12.13 -- excludes one-time model-load cost and PNG decode, reported separately.
- results/sample_dataset/ (32-pair committed example): mean error 2.00px, pass@5px = 0.906.

Robustness Comparison & Ablations

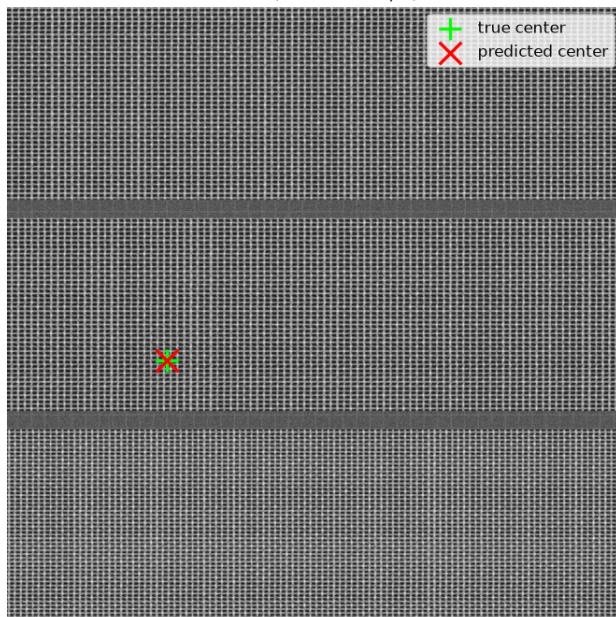
- Baselines (scripts/evaluate_localizer.py): B0 classical ZNCC template matching, B1 10x10 grid search, vs. this model (dense correlation + context head) on identical canvas-disjoint samples -- ZNCC scores misleadingly high confidence even when wrong ~35% of the time; this model's margin-based confidence does not share that failure mode.
- Checkpoint trade-off (an honest, load-bearing finding, not a footnote):

Checkpoint	Pooled mean err	harsh/normal pass@5px	Notes
production_v3 (default)	2.47px	0.640	Consistent across all 4 tested conditions
production_v5 (AFB fine-tune)	41.06px	0.260	100% on real AFB imagery, but lost most harsh-noise robustness

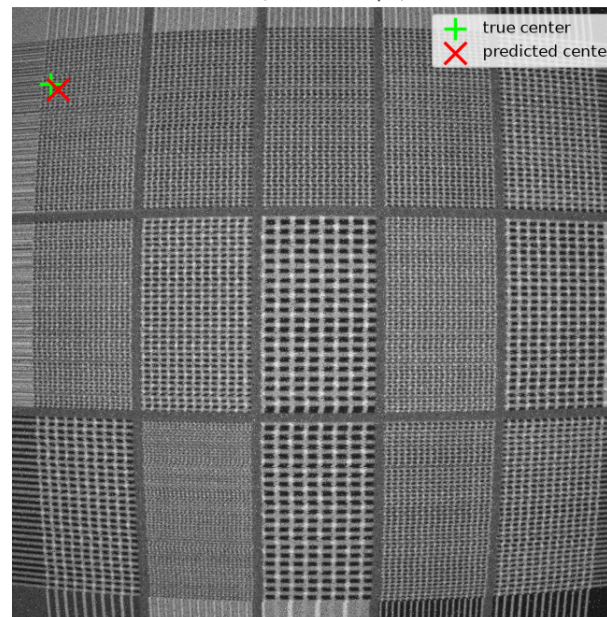
- production_v5, fine-tuned on 5000 real AFB-rendered samples, reaches 100% accuracy at every tested tolerance on an independent 2000-sample AFB test set -- but this same validation matrix shows it lost most of production_v3's harsh-noise robustness in the process. Since sponsor test-data noise characteristics are unknown, production_v3's consistency is the safer default (see README section 4/6).
- A1 ablation (ablation_jitter.py): tests whether the model relies on synthetic noise fingerprints rather than genuine aperiodicity -- a normal->shifted accuracy drop would indicate overfitting to one noise distribution instead of learning aperiodicity in general.

Failure Case, Limitations & Learnings

success case: normal/drift, error=0.24px, confidence=0.879



worst case: harsh/drift, error=16.5px, confidence=0.481



- SUCCESS (left): normal noise, drift geometry -- 0.24px error, confidence 0.879.
- HONEST FAILURE (right): harsh noise, drift geometry -- 16.5px error, confidence 0.481. The model stayed moderately confident despite the large error, suggesting a genuine structural look-alike (periodic-pattern ambiguity) rather than an easily-flagged low-confidence guess.
- Limitations: DRAM-only training (FinFET unwired); harsh-noise conditions remain the hardest case; production_v5 shows a real, measured generalization-vs-robustness trade-off, not yet resolved.
- Learning: catastrophic forgetting appeared twice on different axes -- narrowing training to specific DRAM presets, and fine-tuning on real AFB data -- motivating future work on training that preserves broad robustness while adding new capability.

Conclusion & Next Steps

- Drift-Sense demonstrates that a Siamese dense-correlation model with an explicit global-context head reliably resolves periodic-DRAM-lattice ambiguity that simple template matching cannot -- 2.47px pooled mean error, 22ms median runtime per pair.
- Every structural, noise, and augmentation choice is backed by public sources and documented per-pair generation metadata (references/CITATIONS.md).
- Next steps:
 - Wire `src/patterns/finfet.py` into the training pipeline for genuine dual-architecture support.
 - Investigate mixed real+synthetic fine-tuning to recover `production_v5`'s AFB generalization without losing `production_v3`'s noise robustness.
 - RGB optical-image extension (bonus criterion) once the grayscale SEM task is fully hardened.
- Repository: <https://github.com/nihaltodurkar-cms/puthere>
- Demo video (optional): [VIDEO LINK]